

Анализ задачи распределения ресурсов при помощи конвейеров и маршрутизаторов

Prishelets

2026

Введение

В данной работе рассматривается задача распределения ресурсов при помощи конвейеров и маршрутизаторов в компьютерной игре Satisfactory. Рассматриваются типизация, реализация и взаимодействие друг с другом таких объектов, как: конвейерная лента, конвейерный соединитель, конвейерный разъединитель, и прочие их надстройки. Рассматривается как общее решение задачи распределения ресурсов, так и некоторые частные задачи.

Обоснование выбранного метода: в мире, помимо используемого в основном нами в данной работе метода "балансировки" – ресурсы равномерно распределяются между всеми выходами – существуют и методы "шины" (самого популярного в мире), и даже некоторые экзотические методы, которые я не буду сейчас описывать... "Балансировка" мне нравится тем, что это самый контролируемый метод: строитель чётко знает, где сколько едет, на какую нагрузку что рассчитанно, и так далее. Так же этот метод имеет наибольшую скорость запуска производства: все ресурсы приходят чуть ли не одновременно к потребителям, в то время, как на шине ресурсы заполняют потребителей по одному, начиная с самого начала, а конечные потребители долгое время "голодают".

Contents

1	Терминология	4
1.1	Конвейер	4
1.2	Конвейерный соединитель	5
1.3	Конвейерный разъединитель	6
2	Быстрые системы	7
2.1	Введение	7
2.2	Теорема о числе перестановок	8
2.3	Теорема о делимости	9
2.4	Теорема об упрощении	9
2.5	Алгебра задачи	10
2.5.1	Теория групп	10
2.5.2	Теория графов	11
2.5.3	Линейная алгебра	12
3	Вычисления в быстрых системах	14
3.1	Примеры	14
3.1.1	Упрощение	14
3.1.2	Рекурсия	15
3.1.3	Вариативность	16
3.1.4	Большие схемы	17
3.1.5	Несколько входов	19
3.2	Решение	20

Chapter 1

Терминология

1.1 Конвейер

Конвейер – средство непрерывной перевозки ресурсов между двумя любыми точками.

Каждый конвейер характеризуется:

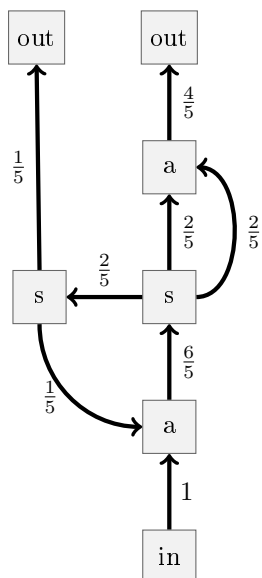
- Формой
- Описывается системой уравнений:

$$\vec{r}(t) = \begin{cases} x_1 = x_1(t) \\ \vdots \\ x_n = x_n(t) \end{cases} \subseteq \mathbb{R}^n$$

В частности: $\mathbb{R}^3$ (Satisfactory) или $\mathbb{R}^2$ (Factoro)

- Функцией распределения динамики скорости вдоль конвейера $v(\vec{r}, t) \cong v(t_{\text{path}}, t_{\text{time}}) = v(\vec{t})$
- Начальной $\vec{r}(0)$ и конечной $\vec{r}(t \rightarrow \max)$ точкой

На чертежах обозначается кривой, либо отрезком, соединяющим пару точек. Может иметь стрелки для указания направления движения, а рядом может подписываться пропускная способность:



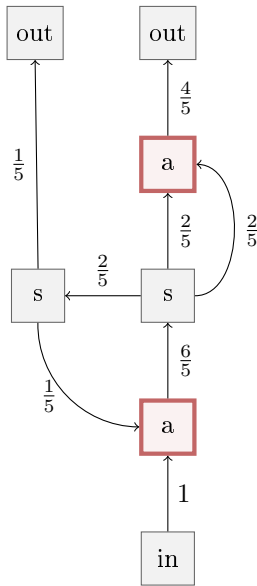
1.2 Конвейерный соединитель

Конвейерный соединитель – устройство, соединяющее несколько входных лент в одну. Ресурсы выводятся по заданным правилам, и выбор между ресурсами производится по другим правилам.

Каждый конвейерный соединитель характеризуется

- Числом входных лент $n \in \mathbb{N}$
- Правилами приёма и распределения: $\{\varphi_i(v_{\text{in}}) \mid i \in \mathbb{N}, i \leq n\}$
- Правилom вывода: ψ

На чертежах обозначается фигурой (обычно, квадратом) с буквой "a" (adder), к которому с одной стороны подводится конвейер на вывод, а с остальных сторон – конвейеры входят:



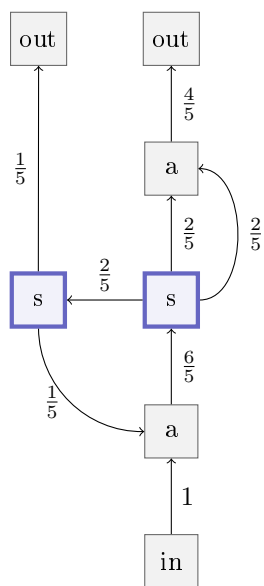
1.3 Конвейерный разъединитель

Конвейерный разъединитель – устройство, разъединяющее одну конвейерную ленту на несколько других, пропускающих ресурсы по заданным правилам.

Каждый конвейерный разъединитель характеризуется:

- числом выходных лент $n \in \mathbb{N}$
- правилами распределения: $\{\varphi_i \mid i \in \mathbb{N}, i \leq n\}$

На чертежах обозначается п-угольником (обычно, квадрат) с буквой "s" (splitter), к которому с одной стороны подводится конвейер, а с остальных сторон – конвейеры отводятся:



Chapter 2

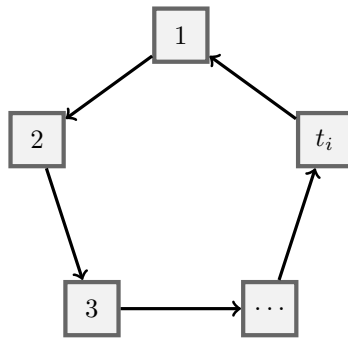
Быстрые системы

2.1 Введение

Рассмотрим следующую систему:

- Конвейеры имеют всюду равномерную скорость v , притом достаточно большую
- Конвейерные разъединители работают по следующим правилам:
 - Всевозможные их числа выходов образуют множество $\mathbb{T} = \{t_1, \dots, t_n\}$
 - Правило вывода: чередовать ресурс между выходами

Схема очередности:



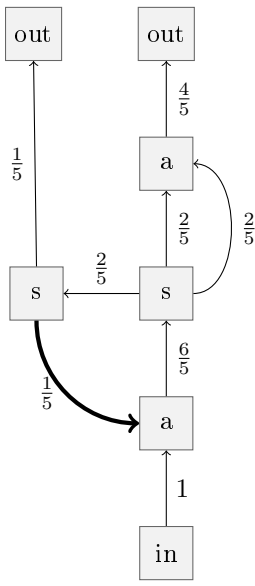
- При этом распределение происходит мгновенно: как только ресурс заходит – так только и выходит
- Конвейерный соединитель может принимать любое число ресурсов, и мгновенно объединяет их в один конвейер

Всвязи с мгновенностью распределений, назовём такую систему "быстрой".

В быстрых системах определён следующий набор действий:

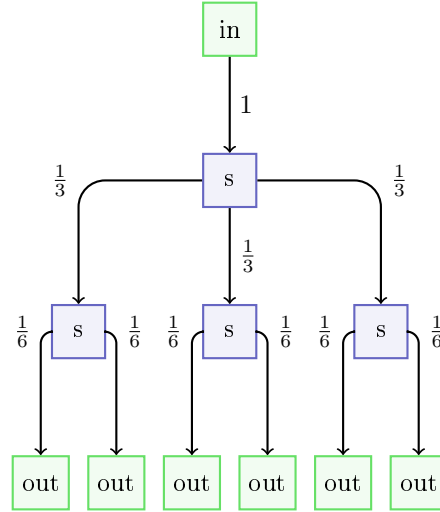
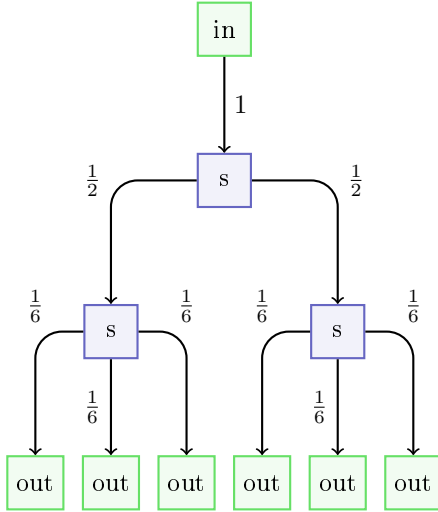
- Разделение – разделение одного конвейера на n равных частей
- Соединение – соединение n конвейеров в один
- Отделение – набор из последовательных и параллельных соединений с целью отделения q_1 ресурсов от конвейерной ленты, перевозящей q_0 ресурсов ($q_1 < q_0$)
- Возврат – сначала отделение некоторой части потока, а затем соединение этой части с началом самого потока

Пример:



Назовём все числа вида $s = \prod_{i=1}^n t_i^{a_i}$, где $a_i \in \mathbb{N}_0$ "стандартными". Тогда каждое число, обратное стандартному¹ может быть получено, как отделение от конвейера в 1 при помощи смеси t_1 -арного, ... , и t_n -арного деревьев.

Пример: для множества $\mathbb{T} = \{2, 3\}$, число $2^1 3^1 = 6$ является стандартным, тогда $1/6$ является единичной дробью, её можно получить двумя способами:



Как можем заметить, дробь одна, а варианта её воспроизводства – два. Отсюда сформулируем теорему:

2.2 Теорема о числе перестановок

Любую единичную дробь, инициализированную числом $s = \prod_{i=1}^n t_i^{a_i}$ можно построить числом способов, равным числу перестановок его сомножителей:

$$P \left[\sum_{i=1}^n a_i \right] (a_1, a_2, \dots, a_n) = \frac{\left[\sum_{i=1}^n a_i \right]!}{\prod_{i=1}^n (a_i!)}$$

¹Будем называть такие числа единичными стандартными, или просто единичными. А если в числителе стоит не единица – то будем называть их стандартными долями.

2.3 Теорема о делимости

Если некоторую стандартную долю конвейера $\frac{a}{s}$ ($a < s$) возвратить в его начало – то на участке до начала отделения пойдёт $q_0 + \frac{a}{s} \cdot q_0$, после второй рекурсии – $q_0 + \frac{a}{s} \cdot (q_0 + \frac{a}{s} \cdot q_0)$, и так далее.

Таким образом, в конечном итоге по данному участку будет идти:

$$\sum_{i=0}^{\infty} q_0 \cdot \left(\frac{a}{s}\right)^i = q_0 \cdot \frac{1}{1 - \frac{a}{s}} = q_0 \cdot \frac{s}{s - a}$$

Так как a можно выбрать произвольное – то от конвейера можно отделить даже не стандартную долю.

Обозначим операцию преобразования дроби $\frac{a}{s}$ в дробь $\frac{a}{b}$ ($b \leq s$) за:

$$\begin{aligned} \frac{a}{s} - \frac{s-b}{s} &:= \frac{a}{s} \cdot \frac{1}{1 - \frac{s-b}{s}} = \frac{a}{b} \\ \frac{a}{s} - \frac{b}{s} &:= \frac{a}{s} \cdot \frac{1}{1 - \frac{b}{s}} = \frac{a}{s-b} \end{aligned}$$

Теорема: любой единичный конвейер можно разделить на любые два положительных рациональных, в сумме дающих единицу, как рациональные доли входного конвейера при помощи операций отделения и возврата

$$\begin{aligned} &\forall q_0, q_1, q_2 \in \mathbb{Q}_0^+ \ (q_1 + q_2 = q_0) : \\ &\exists a, b, s \in \mathbb{N}_0 \ (a, b \leq s) \ (s - \text{standart}) \mid \\ &q_1 = \frac{a}{s} - \frac{b}{s}, \ q_2 = \frac{s-a-b}{s} - \frac{b}{s} \end{aligned}$$

2.4 Теорема об упрощении

Назовём системы:

- "простыми", если: $\forall t_i \in \mathbb{T} : t_i - \text{простое}$
- "полупростыми", если: $\forall t_i, t_j \in \mathbb{T} : \gcd(t_i; t_j) = 1$
- "дублирующими", если: $\exists t, a, b \in \mathbb{T}, \exists x, y \in \mathbb{N}_0, t = a^x b^y$

Теорема: любую дублирующую систему можно упростить:

$$\begin{aligned} &\forall \mathbb{T} \mid \exists t, a, b \in \mathbb{T}, \exists x, y \in \mathbb{N}_0, t = a^x b^y : \\ &\exists \mathbb{T}' = \mathbb{T} \setminus \{t : \exists a, b \in \mathbb{T}, \exists x, y \in \mathbb{N}_0 \mid t = a^x b^y\} \neq \emptyset \\ &\mathbb{T}' - \text{prime, or half-prime} \end{aligned}$$

Таким образом не имеет смысла отдельно изучать дублирующие системы, ведь для них будут верны все свойства полупростых (и простых соответственно). Однако сами по себе дублирующие системы могут быть удобны в решении отдельных задач: например, если в $t = \prod_i t_i^{a_i}$, $\sum_i a_i$ – достаточно большое – то:

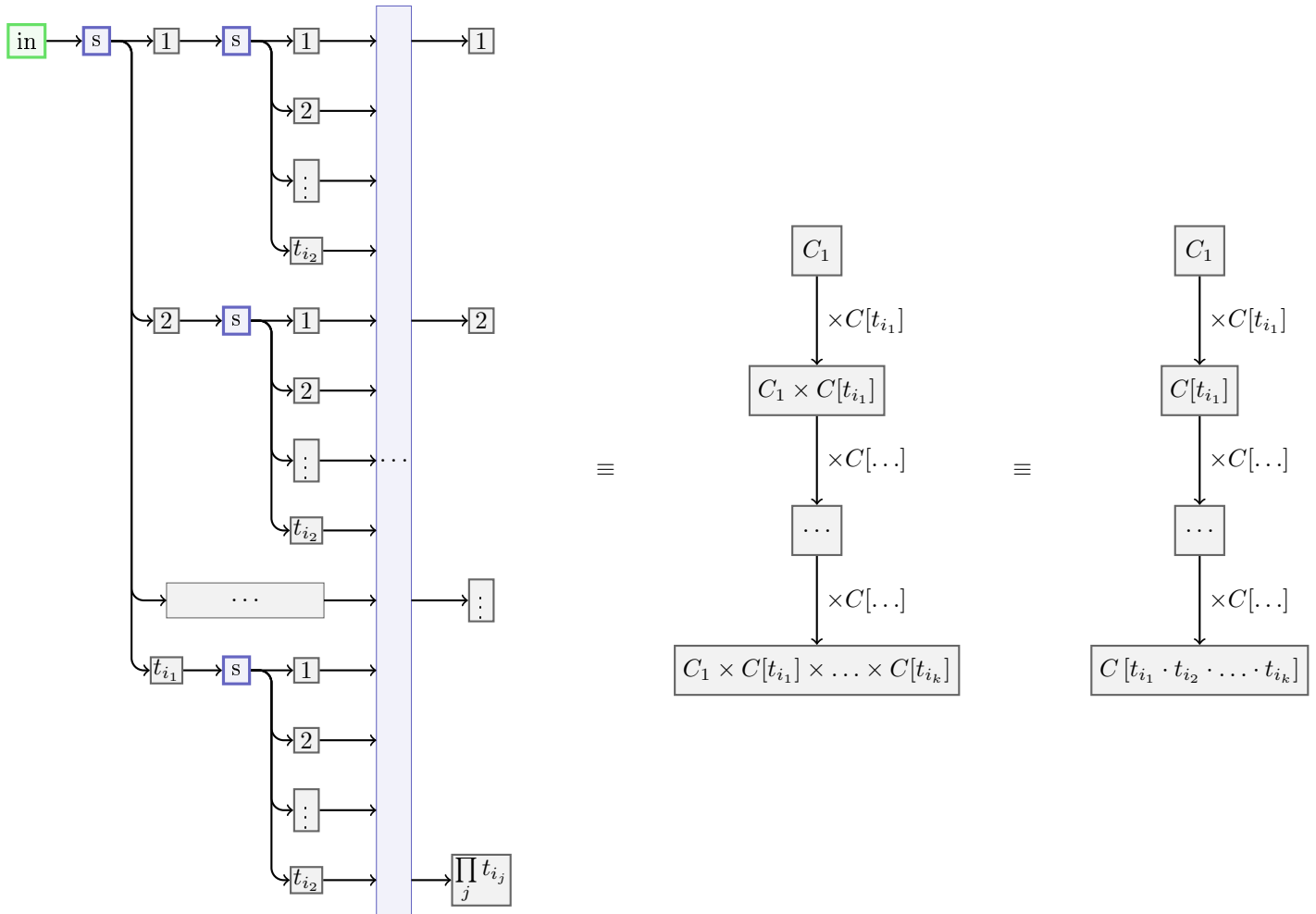
1. Система, реализующая единичную дробь, инициализированную данным числом, будет банально громоздкой
2. Возможных способов реализации такой системы тоже будет много, что значительно усложняет решение задачи

Тогда действительно проще установить готовый разветвитель, а не строить его с нуля.

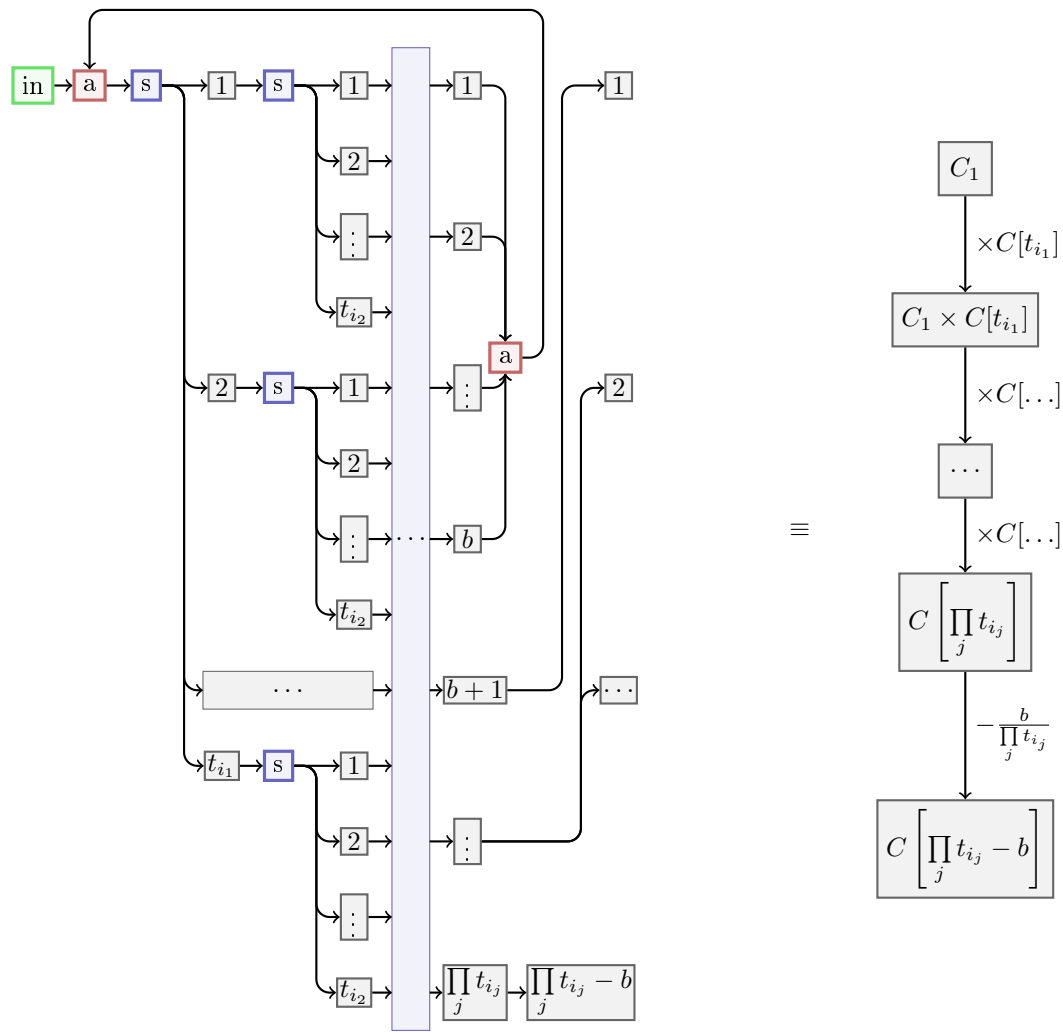
2.5 Алгебра задачи

2.5.1 Теория групп

Так же хочется отметить, что каждый разделитель образует циклическую группу C_{t_i} – тогда резонно сказать, что конвейер в начале своего пути является циклической группой C_1 , а каждый раз, применяя к нему операцию разделения при помощи серии параллельных сплиттеров, мы, по сути, домножаем группу конвейера на группу сплиттера:



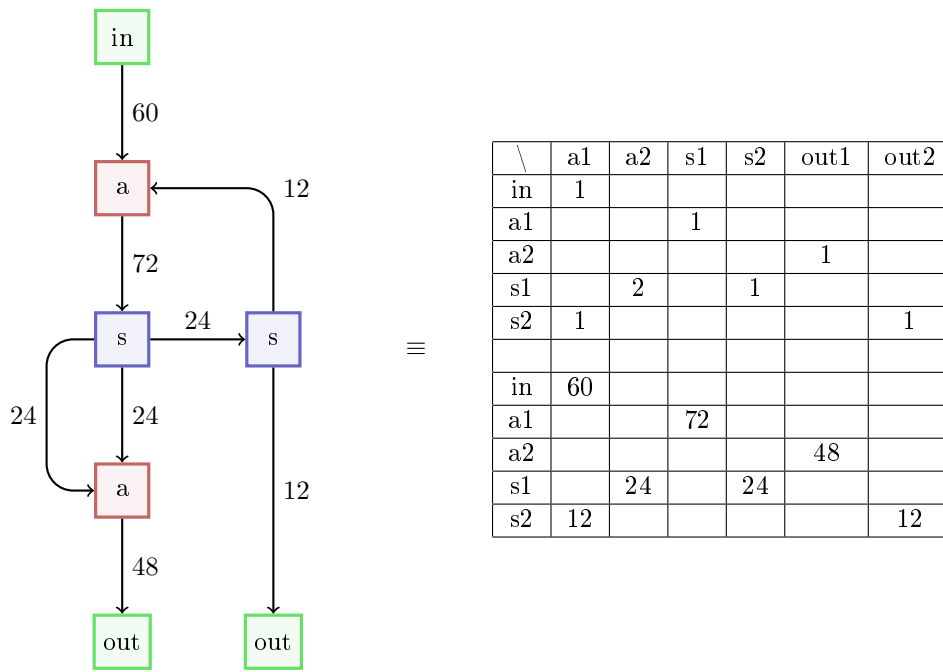
Тогда всё множество выходных конвейеров образует циклическую группу $C[\prod_j t_{i_j}] = C[\prod_i t_i^{a_i}]$. Если из них принудительно b штук вернуть в начало – то мы снова придём к теореме о делимости, и получим уменьшение периода группы: $C[s] \xrightarrow{b} C[s - b]$



Схемы, приведённые справа являются биективными для любой неупрощённой диаграммы разделения, но как мы увидим далее, многие, первоначально кажущиеся сложными и громоздкими схемы – можно довольно сильно сокращать, но тогда мы не сможем рисовать к ним данные схемы так просто. Тем более, такие схемы можно рисовать только для схем *разделения*, но не *отделения*.

2.5.2 Теория графов

Также хочется отметить, что, поскольку каждая схема является мультиорграфом – то её можно описывать не в виде чертежа, а, например, в виде матрицы смежности:



Таблицу, удобства ради, давайте разделим на две части: сверху отметим обычную матрицу смежности для мультиорграфа: число в ячейке говорит о том, сколько рёбер идёт из одной вершины в другую. А снизу – пропускную способность одного ребра.

Такой граф состоит из одной компоненты слабой связности, а все её компоненты сильной связности – это области, благодаря которым происходит операция возврата. Посмотрев на полустепень исхода каждого из узлов компоненты сильной связности, можно выяснить, какой коэффициент усиления она реализует.

Однако матричный метод не является удобным, если все элементы T – малы, а сам граф – большой. Эффективность он приобретает, когда число выходных лент большое, а кратных рёбер при этом мало.

2.5.3 Линейная алгебра

Рассмотрим задачу перераспределения ресурсов между n входами и k выходами: схему перераспределения можно представить в виде матрицы:

$$\begin{bmatrix} m_{11} & \dots & m_{1n} \\ \vdots & \ddots & \vdots \\ m_{k1} & \dots & m_{kn} \end{bmatrix} \times \begin{bmatrix} \text{in}_1 \\ \vdots \\ \text{in}_n \end{bmatrix} = \begin{bmatrix} \text{out}_1 \\ \vdots \\ \text{out}_k \end{bmatrix}$$

Рассмотрим матрицу $A := a_{ij} = \frac{\text{out}_i}{\text{in}_j}$. В ней можно выделить следующие классы значений:

- $a_{ij} = 1$
В этом случае конвейер надо просто провести прямо, без промежуточных преобразований
- $a_{ij} < 1$
В этом случае можно отделить соответствующую долю, а остаток станет новым входом, либо можно сохранить на следующие итерации.
- $a_{ij} > 1$
В таком случае итоговый конвейер получится, как сложение данного с некоторыми другими (или преобразованиями над ними)

Если в такой матрице находится единичный элемент – то в оригинальной матрице на этом же месте мы ставим 1, а остальные значений в тех же столбце и строке – заполняем нулями, а в первоначальной матрице

они удаляются. Среди оставшихся элементов можно выбрать "диагональный" набор дольных значений, и по тем же принципам их занести в итоговую матрицу. Таким образом, должен остаться лишь набор элементов $a_{ij} > 1$. Тогда нам остаётся лишь честно искать решения.

Помочь в этом может следующая теорема: сумма значений в столбцах матрицы преобразования должна давать 1:

$$\begin{bmatrix} m_{11} & \dots & m_{1n} \\ \vdots & \ddots & \vdots \\ m_{k1} & \dots & m_{kn} \end{bmatrix} \times \begin{bmatrix} \text{in}_1 \\ \vdots \\ \text{in}_n \end{bmatrix} = \begin{bmatrix} \text{out}_1 \\ \vdots \\ \text{out}_k \end{bmatrix}$$

$$\sum_{i=1}^k m_{ji} = 1$$

Таким образом, нам остаётся, всего-то навсего, решить систему $n + k$ уравнений и nk переменных:

$$\begin{cases} m_{11} \cdot \text{in}_1 + \dots + m_{1n} \cdot \text{in}_n = \text{out}_1 \\ \vdots \\ m_{k1} \cdot \text{in}_1 + \dots + m_{kn} \cdot \text{in}_n = \text{out}_k \\ m_{11} + \dots + m_{k1} = 1 \\ \vdots \\ m_{1n} + \dots + m_{kn} = 1 \end{cases}.$$

Как известно, такая система имеет единственное решение только если $nk = n + k$, что имеет только две пары целочисленных решений: $(0, 0)$ и $(2, 2)$. Если же точка находится где-то внутри – то решение тривиально: соединение конвейеров, либо обычное отделение. В противном же случае – решений бесконечно много. Если выходов больше, чем входов – то входы можно банально поделить при помощи имеющихся у нас разделителей, и тогда, в какой-то момент мы сможем отделить все требуемые числа. Если входов больше, чем выходов – то наоборот, надо соединять.

Chapter 3

Вычисления в быстрых системах

3.1 Примеры

Рассмотрим $\mathbb{T} = \{2, 3\}$

3.1.1 Упрощение

От конвейера в 60 требуется отделить 15 и 45:

$$\frac{15}{60} = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2}, \quad \frac{45}{60} = \frac{3}{4} = 3 \cdot \frac{1}{2} \cdot \frac{1}{2}$$

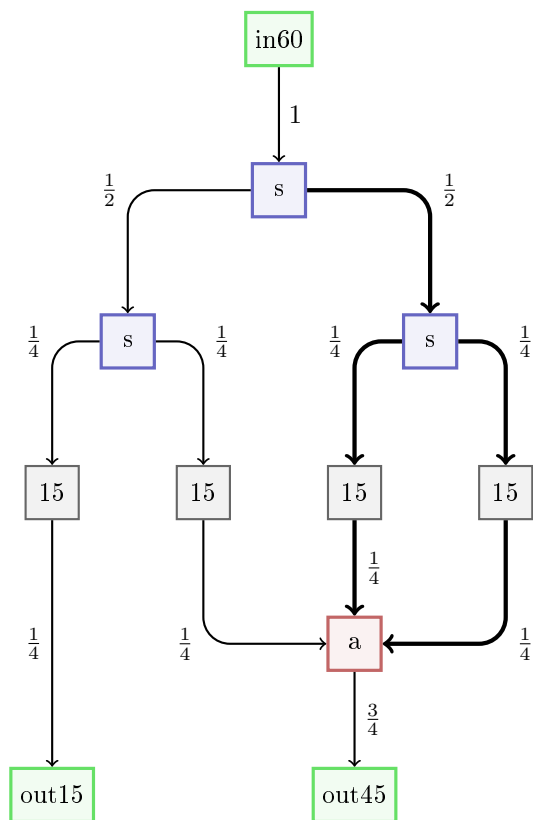


Рис. 1

Заметим, что на рис. 1 имеется **область**, в которой разделение бессмысленно, ведь отделённые друг от друга конвейеры затем вновь соединяются. Тогда можно убрать данный разделитель, оставив целиком **конвейер**.

Для рассмотрения алгебраической картины данной задачи, нам потребуется ввести обозначение: за ответвления в стороны, с припиской в виде числа будем обозначать, что столько-то конвейеров уходят в новую ветку схемы:

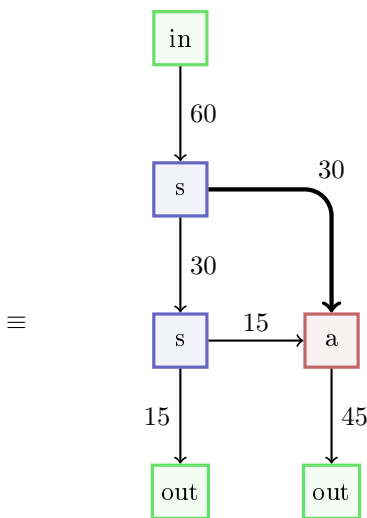
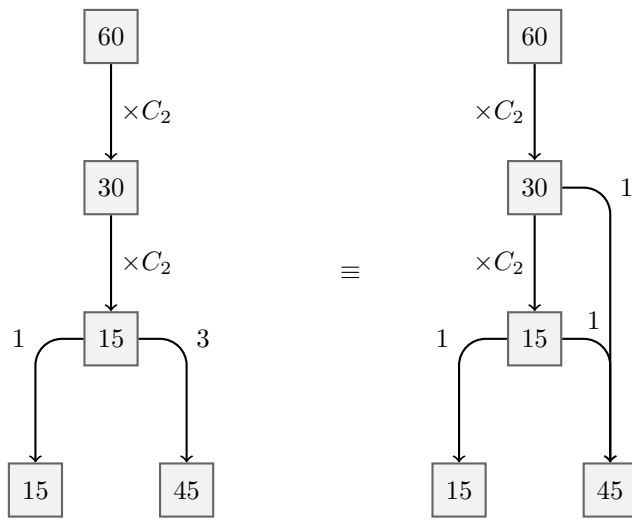


Рис. 2

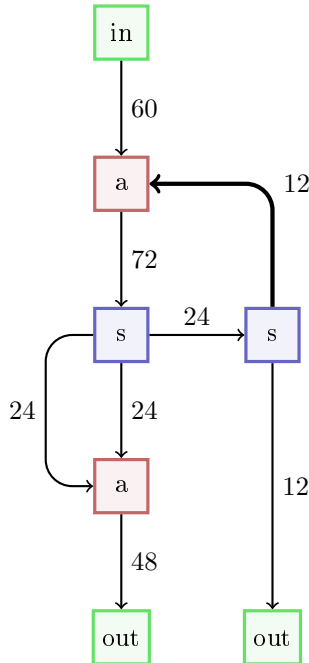


Как видим, при упрощении схемы мы получим, что число конвейеров, идущих на соединение не превышает период циклической группы последнего сплиттера: это будет важным моментом, когда мы будем писать программное решение к задаче.

3.1.2 Рекурсия

От конвейера в 60 требуется отделить 12

$$\frac{12}{60} = \frac{1}{5} = \frac{1}{6} - \frac{1}{6} = \frac{1}{2} \cdot \frac{1}{3} - \frac{1}{2} \cdot \frac{1}{3}, \quad \frac{60-12}{60} = \frac{48}{60} = \frac{4}{5} = \frac{4}{6} - \frac{1}{6} = \frac{2}{3} - \frac{1}{2} \cdot \frac{1}{3}$$

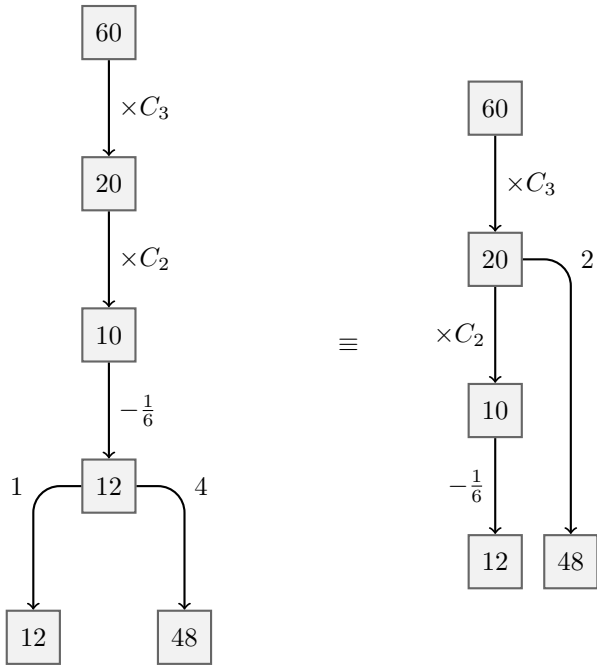


Пусть, в систему импульсами подаётся по одному ресурсу раз в некоторое время¹, тогда внутри делителя на 6 у каждого ресурса будет 6 различных выходов. Положим, первые 5 ресурсов ушли на свои 5 выходов, и как-то распределились в дальнейшей части конструкции. Тогда они поменяли данные, хранящиеся в памяти делителей таким образом, что 6-ой ресурс может уйти только на 6-ой выход. Но этот выход возвращён в начало

¹Такая аппроксимация возможна в связи с тем, что конвейер имеет большую скорость, и ресурсы внутри системы друг с другом не взаимодействуют

системы, и тогда 6-ой ресурс становится 1-ым, так как все настройки делителей сбросились². Таким образом, данный ресурс снова уходит на один из 5-ти допустимых выходов. Притом получается так, что порядок выходов одинаков, так что данная система является равносильной делителю на 5.

Рассмотрим же теперь алгебраическую картину данной задачи:



Забавно, что граф может разбиваться на несколько, казалось бы независимых ветвей, но применив операцию возврата в одной ветви – автоматически меняется значение и во всех других ветвях.

3.1.3 Вариативность

От конвейера в 60 требуется отделить 6. Рассмотрим все (сокращённые) варианты такой схемы:

$$\frac{6}{60} = \frac{1}{10} = \frac{1}{12} - \frac{2}{12} = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{3} - \frac{1}{6}$$

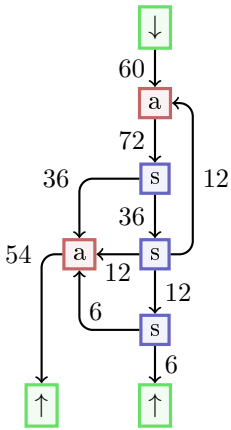


Рис. 1

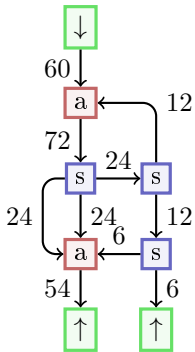


Рис. 2

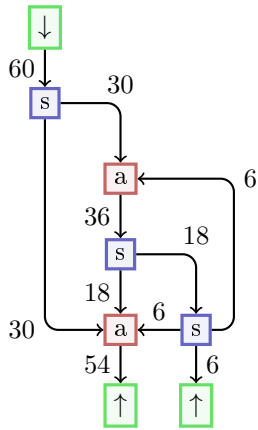


Рис. 3

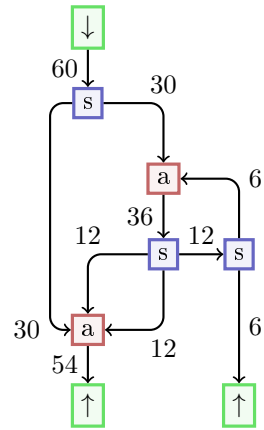
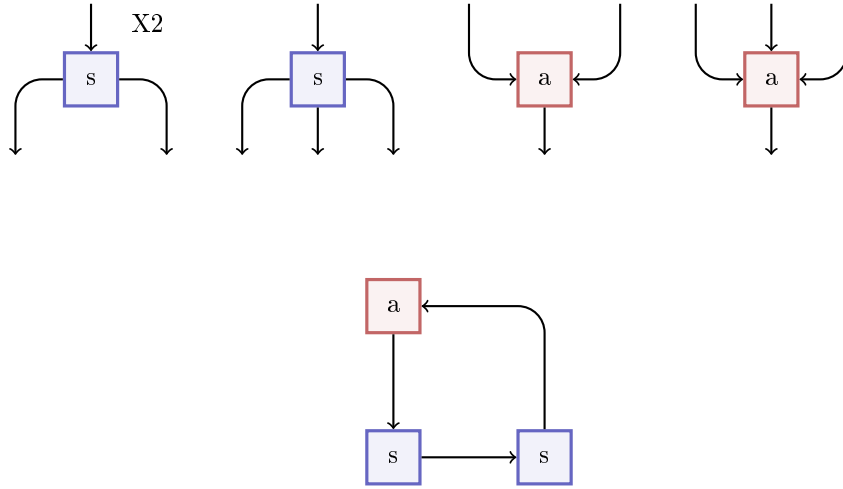


Рис. 4

² Данная система является примером группы $C_2 \times C_3 = C_6$, в которой 6-ой элемент является и 0-ым

Заметим, что в во всех схемах, являющихся сокращёнными вариациями реализации одного и того же решения, число соединителей и разъединителей – инвариантно относительно топологии графа. Притом сохраняется ещё и число входов/выходов конкретного узла, а так же сохраняются подструктуры по типу циклов:



Так же важно помнить, что это всё было только одно решерние: $s = 12$, а ведь можно было взять первые пару ближайших стандартных чисел $(12, 16, 18, \dots)$, или стандартные числа, являющиеся чисто степенями $(16, 32, 27, \dots)$. Так же в решении можно было использоавть нетривиальные разложения по типу: $\frac{5}{6} = \frac{1}{2} + \frac{1}{3}$

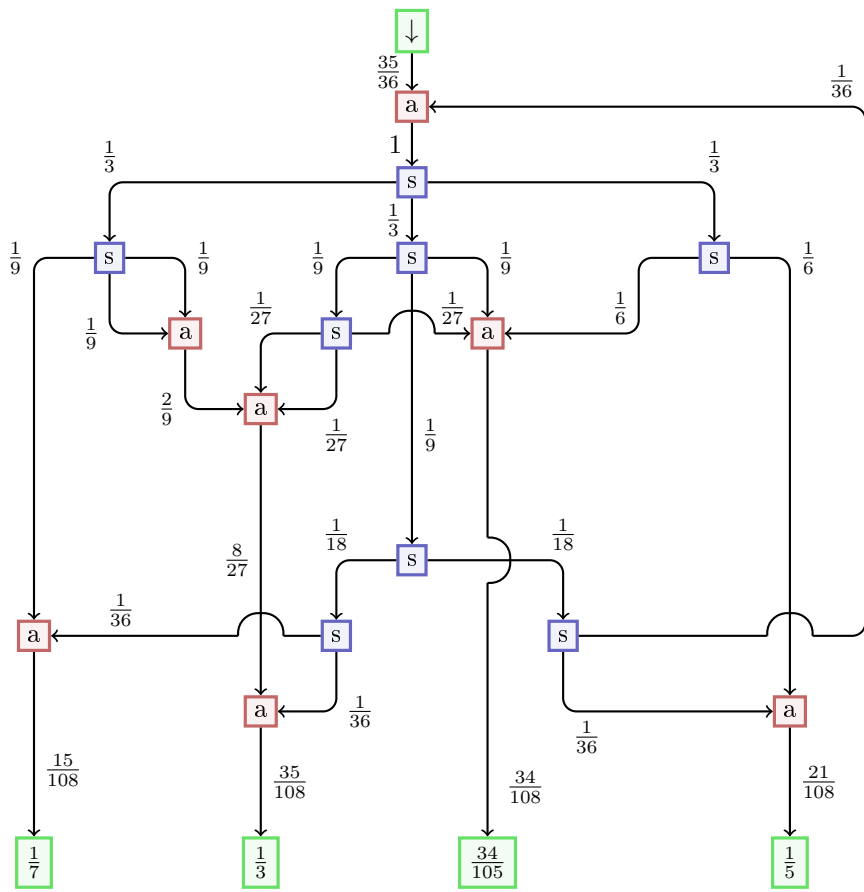
3.1.4 Большие схемы

Рассмотрим схему побольше: отделить $\frac{1}{7}$, $\frac{1}{5}$ и $\frac{1}{3}$ одновременно от одного конвейера.

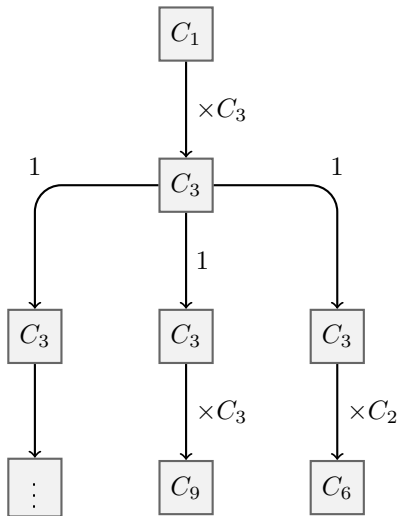
$$\text{lcm}(3, 5, 7) = 105$$

$$\begin{aligned} \frac{1}{7} &= \frac{15}{105} = \frac{15}{2^2 3^3} - \frac{3}{108} = \frac{5}{36} - \frac{1}{36} = \frac{1}{36} + \frac{1}{9} - \frac{1}{36} \\ \frac{1}{5} &= \frac{21}{105} = \frac{21}{2^2 3^3} - \frac{3}{108} = \frac{7}{36} - \frac{1}{36} = \frac{1}{36} + \frac{1}{6} - \frac{1}{36} \\ \frac{1}{3} &= \frac{35}{105} = \frac{35}{2^2 3^3} - \frac{3}{108} = \frac{1}{36} + \frac{2}{27} + \frac{2}{9} - \frac{1}{36} \end{aligned}$$

$$\text{Остаток: } \frac{34}{105}$$



Как мы можем заметить, на схемах побольше появляются две интересные вещи: пересечения конвейеров³ и разная периодичность параллельных ветвей:



Как мы видим, циклические группы C_9 и C_6 находятся в дереве деления на одинаковых уровнях. Это и есть пример применения нетривиальных разложений. Давайте же тогда рассмотрим некоторые разбиения, например, дроби $\frac{34}{108}$:

³Смотреть: проблема перекрёстков (brick factory problem)

$$\begin{aligned}\frac{34}{108} &= \frac{17}{54} = \frac{1}{54} + \frac{8}{27} = \frac{1}{54} + \frac{2}{27} + \frac{2}{9} \\ &= \frac{1}{27} + \frac{5}{18} = \frac{1}{27} + \frac{1}{18} + \frac{2}{9} \\ &= \frac{1}{27} + \frac{1}{9} + \frac{1}{6}\end{aligned}$$

3.1.5 Несколько входов

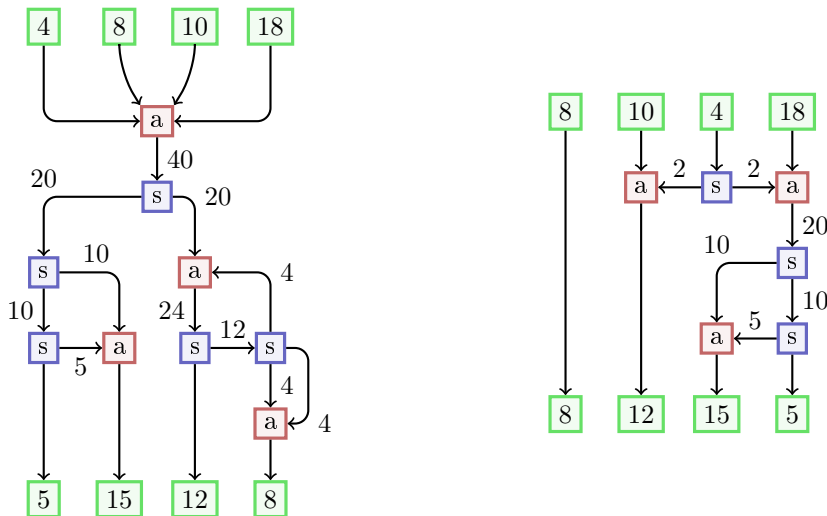
Порой бывает такое, что мы имеем несколько входных конвейеров, идущих, например, с разных производств, и нам необходимо как-то перераспределить между ними ресурсы.

Можно пойти двумя путями: по принципу чайника⁴ : соединить все входные конвейеры, и работать с тем, что получим; или поиграться с отделениями в самих конвейерах:

Пример:

Входы: 4, 8, 10, 18

Выходы: 5, 8, 12, 15



Очевидно, что вторая схема проще первой. Хотя, на самом деле, это не всегда так, и нам просто повезло. Иногда эти две схемы могут быть вообще одинаковыми (например от (10, 10, 10, 10) отделить (5, 8, 12, 15)).

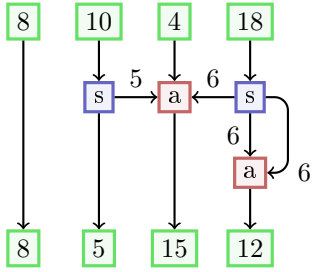
Однако даже так, её можно упростить: как нам уже известно, такая схема является матрицей преобразования вида:

$$\begin{bmatrix} 5 \\ 8 \\ 15 \\ 12 \end{bmatrix} = \begin{bmatrix} m_{11} & \dots & m_{14} \\ \vdots & \ddots & \vdots \\ m_{41} & \dots & m_{44} \end{bmatrix} \times \begin{bmatrix} 10 \\ 8 \\ 4 \\ 18 \end{bmatrix}$$

И имеет решение, например:

⁴Как физик заваривает чай, если чайник пустой? Наливает воду, кипитит воду, заваривает чай.
 Как математик заваривает чай, если чайник пустой? Наливает воду, кипитит воду, заваривает чай.
 Как физик заваривает чай, если в чайнике есть вода? Кипитит воду, заваривает чай.
 Как математик заваривает чай, если в чайнике есть вода? Выливает воду, наливает воду, кипитит воду, заваривает чай.
 Принцип чайника: сведи задачу к предыдущей.

$$\begin{bmatrix} 5 \\ 8 \\ 15 \\ 12 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \frac{1}{2} & 0 & 1 & \frac{1}{3} \\ 0 & 0 & 0 & \frac{2}{3} \end{bmatrix} \times \begin{bmatrix} 10 \\ 8 \\ 4 \\ 18 \end{bmatrix}$$



Таким образом, мы получили самую простую версию схемы перераспределения ресурсов между несколькими входами и выходами.

3.2 Решение

Любую стандартную долю можно представить, как сумму некоторого количества других стандартных дробей:

$$\forall \frac{a}{s} : \exists n \in \mathbb{N}, A = \{a_i\}, S = \{s_i\} \mid \frac{a}{s} = \sum_{i=1}^n \frac{a_i}{s_i}$$

"Элементарными" назовём несократимые стандартные дроби, для которых $n = 1$, т.е.:

$$\nexists a_i \leq a \mid \gcd(a_i, s) \neq 1$$

Для дроби $\frac{a}{b}$ "решением" назовём⁵ $A_b^a = O_b^a - R_b$, где:

- $s = s(b)$ – ближайшее большее стандартное к b
- $O_b^a = O_s^a = \sum_{i=1}^n \frac{a_i}{s_i}$
- $R_b = O_s^{s-b}$

Тогда задача сводится к изучению получения оптимальных $S(b)$, O_s^a и R_b .

Код на C# для реализации поиска наименьшего $s(b)$ для множества $\mathbb{T} = \{2, 3\}$:

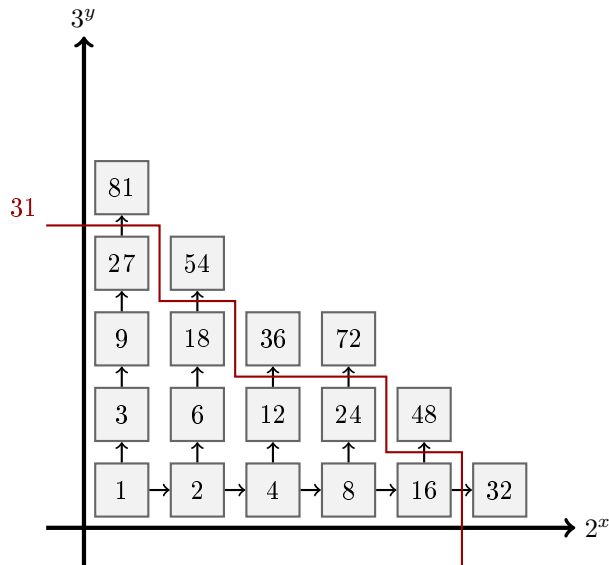
⁵под "-" подразумеваем его определение через рекурсию

```

1 public static int hig_closest(int number, int x = 0, int y = 0, int branch = 1)
2 {
3     int answer = -1;
4     int num = Convert.ToInt32(Math.Pow(2, x) * Math.Pow(3, y));
5
6     if (num >= number) { answer = num; }
7     else
8     {
9         List<int> answers = new List<int>();
10
11         answers.Add(hig_closest(number, x + 1, y, 0));
12         if (branch == 1)
13             { answers.Add(hig_closest(number, x, y + 1, 1)); }
14
15         answer = answers.Min();
16     }
17
18     return answer;
19 }

```

Он реализует следующее дерево поиска (пример для $b = 31$):



Нетрудно модифицировать данный алгоритм для произвольных $\mathbb{T}$:

```

1 def hig_closest(target, powers, branch=max(T)):
2     answer = -1
3     num = 1
4     for i in range(0, T.len()):
5         num *= T[i]^powers[i]
6
7     if num >= target: return num
8
9     answers = []
10    for i in T:
11        if i <= branch:
12            answers.append(hig_closest(target, powers.increase(i), i))
13
14    answer = min(answers)
15
16    return answer

```

Вот нашли мы $s(b)$, а как теперь искать O_s^a ? Ответ прост: брутфорс. Требуется перебрать все возможные способы разбиения числа a на суммы всевозможных натуральных⁶ чисел, после чего соответствующие дроби (если это возможно) мы сокращаем дроби, а затем объединяем все дроби с одинаковыми знаменателями. Для каждого отдельного знаменателя $s_i = \prod_{j=0}^n t_j^{a_j}$ должно быть выполнено: $s_i < t_j \forall j$

Алгоритм для поиска одного варианта разбиения:

```

1 def splitter(numerator, denominator):
2     d_c = hig_closest(denominator)
3     d_g = gcd(d_c, numerator)
4     d_d = numerator // d_g
5     d_f = d_c // d_g
6
7     d_p0 = low_closest(d_d, d_f)
8
9     if d_p0 == d_d:
10        return [[d_d, d_f]]
11
12    else:
13        g1 = gcd(d_d-d_p0, d_f)
14        g2 = gcd(d_p0, d_f)
15        d_parts = [[(d_d-d_p0)//g1, d_f//g1], [d_p0//g2, d_f//g2]]
16
17    while 1:
18        d_parts_ = []
19
20        for fractions in d_parts:
21            x, y = fractions[0], fractions[1]
22            p = low_closest(x, y)
23
24            if p != x:
25                g1 = gcd(x-p, y)
26                g2 = gcd(p, y)
27                d_parts_.append([(x-p)//g1, y//g1])
28                d_parts_.append([p//g2, y//g2])
29
30            else: d_parts_.append(fractions)
31
32        if len(d_parts) == len(d_parts_): break
33        else: d_parts = d_parts_[0::]
34    return d_parts[0::]

```

Полноценный алгоритм на C# приведён в satisfactory_conveyor_calc.sln

⁶В целом, можно и не натуральных, а просто рациональных: $\frac{2}{3} = \frac{1.5}{3} + \frac{0.5}{3} = \frac{1}{2} + \frac{1}{6}$ – такое разбиение вполне подойдёт для $\mathbb{T} = \{2, 6\}$. Однако, такой алгоритм слишком сложен, ведь перебирать придётся **значительно** больше вариантов